NETRAX · AXIS P1447-LE camera setup

Real IP CCTV camera on the Welding bay + new camera plan · steps for us and prompts for Claude Code · 13 Nov 2026

Note for Claude Code

Do **only** the prompts in Part 4, one at a time, in order. Stop after each and show the result. Parts 2–3 are done by the team by hand (cable, network, password). Never write or print the camera password; it lives only in `config/secrets.local.yaml`.

1 · The new camera plan

ID	Place (zone)	Camera	Source in cameras.yaml	Connection
cam_01	Welding bay	**AXIS P1447-LE** (IP CCTV, 5 MP, IR)	`rtsp://{axis}@192.168.0.90/axis-media/media.amp?…`	Network cable + PoE
cam_02	Warehouse	Laptop built-in webcam	`0` (check index)	Built in
cam_03	Assembly line	Phone, IP Webcam app	`http://PHONE-IP:8080/video`	Wi-Fi
cam_04	Main entrance	USB webcam (unchanged)	`1` (check index)	USB

The camera IDs stay the same, so zones, required PPE, restricted-zone rules, history, map and reports keep working. Only the *source* of each camera changes. The old file is kept as `config/cameras.before_axis.yaml`, and `use_cameras.bat old` brings the old setup back in one step.

How the Axis picture reaches NETRAX

AXIS camera → PoE injector → laptop Ethernet (192.168.0.20) → RTSP video (TCP, 1280×720, 15 fps) → NETRAX edge AI → dashboard, Telegram, voice. The internet stays on the laptop's Wi-Fi.

The AI runs on the laptop next to the camera, so there is no cloud delay. This is the same edge idea we explained to the jury, now with a real CCTV camera.

Why these settings

- **1280×720 at 15 fps**, not 5 MP: enough for PPE, light for the GPU and the other 3 cameras.
- **RTSP over TCP:** no broken grey frames.
- **View-only user**, password only in the secrets file.
- **No gateway on Ethernet**, so the internet still works.

2 · What we do by hand first (about 20 min)

Bring: the camera, the **PoE injector or PoE switch** your friend used, a network cable, a USB-to-Ethernet adapter if the laptop has no Ethernet port, and the camera's admin password from your friend.

#	Step (done by a person)	How
A	Power and cable	The P1447-LE has no power socket: it is powered over the network cable (**PoE**). Use the same PoE injector or PoE switch your friend used. Camera → PoE injector "PoE / Data+Power" port; injector "Data / LAN" port → laptop Ethernet port (or a USB-to-Ethernet adapter). Wait about 1 minute for the camera to start.
B	Give the laptop an address on the camera network	192.168.0.90 is the address an Axis camera uses when nobody gives it one, so the laptop needs a fixed address in the same range. Windows: Settings → Network & internet → Ethernet → IP assignment → Edit → Manual → IPv4 on: IP `192.168.0.20`, subnet `255.255.255.0` (prefix 24), gateway and DNS **empty**. Wi-Fi stays on for the internet (Telegram, e-mail, phone camera).
C	Check the connection	Command Prompt: `ping 192.168.0.90` → replies. Or install AXIS IP Utility on our laptop too: the camera must appear in its list.
D	Log in and make a view-only user	Browser: `http://192.168.0.90` → log in with the admin user and password your friend set. System → Users (Accounts) → Add: user `netrax`, a password, role **Viewer**. Check that RTSP is not disabled (System → Network / Plain config: RTSP port 554). Note the image settings: 16:9 capture mode if you can choose.
E	Test the video in VLC	VLC → Media → Open Network Stream → `rtsp://netrax:PASSWORD@192.168.0.90/axis-media/media.amp?videocodec=h264&resolution=1280x720&fps=15` (use the real password). The live picture must appear within 2–3 s. If not, see Part 5.

F	Put the password in our secrets file	Open `config\secrets.local.yaml` yourself and add (after Prompt 1 is done):
		`cameras:`
		`  axis: {user: netrax, password: "your-password"}`
		Do not paste the password into the Claude Code chat.

G	Phone for the assembly line	Phone on the same Wi-Fi as the laptop, IP Webcam app → Start server. Note the IP it shows (http://IP:8080). Keep the phone on a charger.
H	Mount the cameras	Axis → welding bay, 2.5–3 m high, looking down about 20–30°, whole worker (head to shoes) in the picture. Laptop webcam → warehouse view. USB → entrance (as before). Phone → assembly line, upright on a stand.

Check before going on

Steps A–E done and the Axis video plays in VLC on **our** laptop. If VLC cannot show it, NETRAX cannot either. Fix it with Part 5 first.

3 · Safety of our original work

- Every file is copied to `_to_delete/before_axis_13nov/` before it changes, plus `config/cameras.before_axis.yaml`.
- No model, weight, threshold, dashboard or notification change. Only camera reading and camera config.
- `python -m edge.selftest` after each prompt; local commit only after the full test works (Prompt 4); no push.

4 · Prompts for Claude Code (VS Code)

Total about 2 hours. Paste one prompt, wait, do the check, then the next.

Prompt 0 · Read, back up, report (no changes) — 10 min

```
Read CLAUDE.md first. We are changing ONLY the camera setup of NETRAX. Do not touch models/,
weights, training, detection thresholds, the dashboard design or notifications.

New camera plan (camera IDs stay the same, so zones, history, map and reports keep working):
  cam_01  Welding bay    -> AXIS P1447-LE IP CCTV camera, RTSP, IP 192.168.0.90 (wired, laptop
Ethernet)
  cam_02  Warehouse      -> LAPTOP built-in webcam (today it is a phone)
  cam_03  Assembly line  -> PHONE with the IP Webcam app (today switched off)
  cam_04  Main entrance  -> USB webcam, unchanged (role: entrance)

Rules for the whole job:
- Before changing any file copy it to _to_delete/before_axis_13nov/<same path>. Also copy
config/cameras.yaml to config/cameras.before_axis.yaml (this is our one-step fallback).
- The Axis user name and password must NEVER be written in cameras.yaml, code, logs, screenshots
or git. They go only in config/secrets.local.yaml (git-ignored). Mask them as *** whenever a URL
is printed.
- Keep every existing feature working. After each step run:  python -m edge.selftest   (all
checks must pass).
- Never commit config/secrets.local.yaml, config/users.local.yaml, firmware/*/secrets.h or
data/. Do not push.

Now do only this (no code changes yet):
1. Make the backups above.
2. Read edge/stream.py, edge/phones.py, tools/find_phones.py, tools/camdiag.py, edge/run_edge.py
(or wherever cameras.yaml is loaded), config/cameras.yaml, config/secrets.example.yaml.
3. Tell me in a short list: how an rtsp:// source is opened today, whether OpenCV here was built
with FFmpeg (print cv2.getBuildInformation() Video I/O part), how the phone auto-follow
(find_phones --write, phones.register) decides which camera is a phone, and anything that would
break if cam_02 becomes a webcam index.
```

Check before the next prompt:

☐ Backups and config/cameras.before_axis.yaml exist

☐ You got the short report

Prompt 1 · RTSP support: secret login, TCP, low delay, test tool — 35 min

```
Step 1 - make RTSP cameras safe and fast in edge/stream.py (no camera change yet).

1. Credentials from secrets: support a source written as  rtsp://{axis}@192.168.0.90/axis-
media/media.amp?...  where {axis} is replaced at start-up by "user:password" from
config/secrets.local.yaml:
     cameras:
       axis: {user: netrax, password: "..."}
```

```
   URL-encode user and password (special characters!). Add the same block with empty values to
config/secrets.example.yaml. If the secret is missing, the tile shows "Axis password not set in
secrets.local.yaml" instead of crashing.
2. Mask the password (***) in every print, status line, log and API response (/api/cameras or
health) that shows a source.
3. RTSP over TCP and short timeouts: before cv2 is imported, if OPENCV_FFMPEG_CAPTURE_OPTIONS is
not already set, set it to "rtsp_transport;tcp|stimeout;5000000" (TCP = no grey / broken frames
on a busy network). Open rtsp sources with cv2.VideoCapture(url, cv2.CAP_FFMPEG).
4. Low latency: for network sources keep only the newest frame (the reader thread must grab
continuously and overwrite; BUFFERSIZE does not work for FFmpeg). Measure and print once per
minute: camera fps and frame age in ms.
5. The quick reachable() check already uses port 554 for rtsp - keep it, and make the OFFLINE
message for rtsp say: "Axis camera not reachable at 192.168.0.90:554 - Ethernet cable / laptop
IP 192.168.0.x?".
6. Add a tool:  python -m tools.rtsp_test cam_01   (or a URL) that opens the stream for 10 s and
prints: opened yes/no, resolution, fps, average frame age, first error in plain words, and saves
one picture to _to_delete/axis_test.jpg.
Add selftest checks: {axis} placeholder is filled; password never appears in the masked text;
missing secret gives the friendly message.
```

Check before the next prompt:

- ☐ selftest passes
- ☐ No password in any file except secrets.local.yaml

Prompt 2 · New camera plan + phone tool + switch script

25 min

```
Step 2 - the new camera plan in config/cameras.yaml (keep the IDs and the comments style).

  cam_01:  name Welding bay,  source "rtsp://{axis}@192.168.0.90/axis-media/media.amp?
videocodec=h264&resolution=1280x720&fps=15",
           default_zone welding_bay, kind: cctv, no rotate.
  cam_02:  name Warehouse,    source 0 (laptop built-in webcam), default_zone warehouse, no
rotate, no phone follow.
  cam_03:  name Assembly line, enabled true, source http://<phone IP>:8080/video (I will give
the IP, or find it with python -m tools.find_phones), rotate 90 if the picture is sideways,
default_zone assembly_line.
  cam_04:  Main entrance, USB webcam, role entrance - unchanged except the index if step 3 shows
it moved.

Also:
1. tools/find_phones.py and edge/phones.py: the phone cameras are now ONLY cam_03 (CAMS dict, --
write, --cam02/--cam03 options). find_phones --write must never overwrite cam_01 (Axis) or
cam_02 (webcam). Keep --cam03; make --cam02 print "cam_02 is now the laptop webcam".
2. Webcam indices: run python -m tools.camdiag (or a 2-second test of indices 0,1,2 with
CAP_DSHOW) and save one picture per index into _to_delete/ so I can see which index is the
laptop camera and which is the USB camera. Ask me before writing the indices if it is not clear.
3. Make a small switch script  use_cameras.bat  with two choices:
     use_cameras.bat axis   -> config/cameras.yaml = the new plan (keep a copy as
config/cameras.axis.yaml)
     use_cameras.bat old    -> copies config/cameras.before_axis.yaml back
   It only copies files; then the user restarts run_all.bat.
4. Update the header comments of cameras.yaml: source can also be an RTSP CCTV camera with
{axis} credentials from secrets.local.yaml.
Do not start the system yet; show me the final cameras.yaml (password-free) and the camdiag
pictures.
```

Check before the next prompt:

- ☐ cameras.yaml has {axis}, not the password
- ☐ You know which index is laptop / USB

Prompt 3 · Test all 4 cameras and the AI

30 min

```
Step 3 - test with all 4 cameras (the Axis camera is connected by cable and the phone has IP
Webcam running).

1. python -m tools.rtsp_test cam_01  -> must open, 1280x720 (or the size the camera gives),
about 15 fps, frame age under 300 ms. If the camera refuses 1280x720, try resolution=1280x960,
then 1920x1080 with fps=10, and keep the first that works; tell me which.
2. Start the system (run_all.bat). Check every tile on the control room: name, LIVE, fps. Read
the edge window status lines for all 4 cameras for 2 minutes.
3. AI check on the Axis camera: one person in the welding bay without helmet -> PPE alert on
cam_01 / Welding bay with the right worker; with all PPE -> no alert. Restricted zone rule
(allowed_workers in zones.yaml) still works.
4. Measure: per-camera fps (camera and AI), AI time per frame, frame -> dashboard delay (we
measured 0.28 s before). Write the numbers in a table. Only measured numbers.
5. If anything is wrong, use the troubleshooting list from the PDF, fix only the camera part,
and tell me what you changed.
6. Run python -m edge.selftest - all pass.
Stop here and show me the results before Step 4.
```

Check before the next prompt:

- ☐ 4 tiles LIVE
- ☐ Helmet test on the Welding bay works
- ☐ Numbers measured

Prompt 4 · Record the work and commit locally

15 min

```
Step 4 - only after Step 3 worked: record the new work.

1. CLAUDE.md: add a short "13 Nov - Axis P1447-LE CCTV" section: camera plan table, how the RTSP
URL and {axis} secret work, laptop Ethernet IP 192.168.0.20/24 without gateway, use_cameras.bat
axis/old, the measured numbers from Step 3.
2. docs/: add docs/AXIS_CAMERA_SETUP.md (same content, written for a new team member) and one
screenshot of the control room with all 4 tiles live (no passwords visible).
3. Check git status: no secrets, no data/, no _to_delete/. Then a LOCAL commit:
   "Axis P1447-LE IP CCTV on Welding bay over RTSP (TCP, secrets-based login), laptop webcam ->
Warehouse, phone -> Assembly line, USB entrance unchanged; rtsp_test tool, use_cameras switch"
   Do not push.
4. Tell me how to go back to the old setup in one line (use_cameras.bat old + restart).
```

Check before the next prompt:

- ☐ CLAUDE.md + docs updated
- ☐ Local commit without secrets

5 · Troubleshooting

Problem	Fix
ping 192.168.0.90 gives no reply	Camera powered? (LED on the camera / injector). Cable into the injector's LAN port, not the PoE port. Laptop Ethernet IP really 192.168.0.20 / 255.255.255.0? Disable VPN. Try AXIS IP Utility: if it shows a different IP, use that IP everywhere.
ping works, browser asks for password, VLC says 401 / unauthorised	Wrong user / password, or the netrax user is not saved. Try the admin user in VLC once to rule it out (never put the admin password in our files).
VLC works, NETRAX tile OFFLINE	Run `python -m tools.rtsp_test cam_01` and read the first error. Usually: secret missing in secrets.local.yaml, a special character in the password (Prompt 1 URL-encodes it), or OpenCV without FFmpeg.
Picture grey, smeared or breaks every few seconds	RTSP over UDP is losing packets: Prompt 1 forces TCP. Check the environment variable is set before OpenCV starts.
Picture 2–5 s late	Old frames piling up: the reader must keep only the newest frame (Prompt 1 step 4). Lower to fps=10 or resolution 1280x720.
“Resolution not supported”	The camera's capture mode decides the sizes: try 1280x960, or 1920x1080 with fps=10.
All cameras get slow after adding the Axis	Use 1280x720 at 15 fps (not 5 MP). The phone at 1 fps → phone on charger, same Wi-Fi band, or laptop hotspot (see our 4-hour plan).
Laptop webcam / USB webcam swapped	Swap source 0 and 1 between cam_02 and cam_04, restart.
Internet stopped after plugging the cable	The Ethernet gateway must be EMPTY, otherwise Windows sends internet traffic to the camera cable.
Need to go back quickly	`use_cameras.bat old` then restart run_all.bat.

6 · What to tell the jury

> **“**We connected a real AXIS P1447-LE IP CCTV camera to NETRAX over RTSP, the same standard used by CCTV recorders. The AI runs on the edge computer on the camera's own network, so the video never goes to the cloud and the delay stays low. Any existing IP CCTV camera can be added the same way: its address in one config line, a view-only login, and it appears on the dashboard with PPE and fire detection.**”**

Axis RTSP address format: `rtsp://user:password@IP/axis-media/media.amp` with optional `videocodec`, `resolution` and `fps` parameters (Axis VAPIX). Sources: axis.com VAPIX library; camstreamer.com RTSP parameters.